Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

THE EVOLUTION OF MACHINE LEARNING AND AI

Week 4 Quiz Summary

Three Ways to Wire a Machine: everything the quiz can ask, in plain English

How to use this page. Week 4 is examined in the **Week 6 quiz** (50% of the module). The quiz is **multiple choice**. Everything it can ask about Week 4 is in the **ten points** below, in the order of the lecture.

Read the boxes (about 20 minutes). Do the questions (about 20 minutes). Mark them. Re-read only the points you got wrong.

If you remember one thing. How you wire a machine is a guess about the world. A true guess is worth a mountain of data; a false one is worse than no guess. With little data the well-guessed machine wins; with enormous data, the machine that guesses less learns the structure itself and then wins.

1 Part 1. The ten points

1.1 Every learner needs a guess

1 goes to 2, 2 to 4, 3 to 6: what does 4 go to? You said 8, but infinitely many rules fit the three examples. Picking one needs a preference that is not in the data: the **inductive bias**. Every learner, human or machine, has one. **How a model is wired**, which inputs may influence which outputs, is such a guess about the world. A true guess means fewer dials, less data and better results on new examples. The three families of the week are three guesses: **convolution** (things near each other belong together), **recurrence** (one thing after another), **attention** (content decides who talks to whom).

1.2 A flexible model is not a better model

From the same 128 pictures, a tiny model with a built-in guess (700 dials) scored **0.945** on new pictures; a big model with no guess (16,708 dials) scored **0.275**, close to chance (0.250), while scoring **1.000** on the pictures it studied. It was not undertrained: it had no reason to prefer the right answer. The **shuffle test** proves where an advantage comes from: destroy only the thing the guess is about (here, which pixels are neighbours) and see who suffers. The tiny model fell to chance; the big one did not move. Its whole advantage was the guess, and when the guess became false it was worse than no guess.

1.3 Convolution: a stencil slid over the picture

A **kernel** is a small square of numbers, usually three by three. Lay it on the picture, multiply each pixel by the number on top of it, add, write one number; slide one step and repeat. **The nine numbers never change as it moves**: that is **weight sharing**. The output is a **feature map**, a map of where the kernel's pattern was found. An edge kernel whose numbers add up to zero gives nothing on a flat patch: it detects **change**, not brightness. Three assumptions: **locality** (nearby things belong together), **stationarity** (a pattern worth finding here is worth finding everywhere) and **compositionality** (big things are made of small things, so you stack layers).

1.4 Equivariance, invariance and the ending

Shift the picture then convolve, or convolve then shift: the results are **exactly** the same (difference 0.00e+00). That is **equivariance**: the output moves with the input. It comes from the arithmetic, not from training. **Invariance** is different: the output does not change at all, which is what you usually want (a cat is a cat wherever it sits). The **ending** decides whether you get it: average the whole map and the model scored **0.998** on positions it never saw; flatten and wire every position separately and it scored **0.230**. A guess is worth only what the ending keeps of it. Stacking layers grows the **receptive field**: by layer 8 one unit sees all 1,024 pixels.

1.5 Where the convolution guess fails

It fails when the data has **no meaningful arrangement** (a spreadsheet, where column order means nothing), when **position matters** (a shadow high in a lung is a different finding from one low down), when the important things are **far apart**, and when the change is **not a shift** (rotation, size). And on pictures that were **already centred and cropped**, the guess bought nothing: the plain model scored 0.978 and the convolution model 0.940. Somebody had already solved "where is it?". Convolution is not "better for images" in general; it is better when position still needs solving.

1.6 Recurrence: one reader, one notepad

Sequences break three things at once: length varies, **order carries meaning** (“dog bites man” against “man bites dog”), and related words can be far apart. A **recurrent** network is one reader with a **fixed-size notepad** (the **hidden state**): read a word, update the notepad with the **same rulebook** every time, move on, answer from the notepad at the end. It is weight sharing again, turned from space to time.

1.7 Why long chains forget, and what gates fix

The signal from word 1 reaches word 60 through sixty identical updates, multiplied by **the same number** each time: 0.9 sixty times is about **0.002**; 1.1 sixty times is about **304**. That is the **vanishing (or exploding) gradient**, and it is arithmetic, not a bug. The **LSTM** (1997) adds a memory with a **keep-gate**, a number between 0 and 1 chosen fresh at every word; the memory is multiplied by the gate and **added to**, not squashed, so the signal can survive hundreds of steps. Two limits remain: the notepad is still a **fixed size** (at 160 words the gated reader’s signal survived and it still failed), and the chain runs **one step at a time**, which suits modern chips badly.

1.8 Attention: a noticeboard, not a corridor

Every word looks at every other word directly, and **content** decides who listens to whom. The guess is **weak**: nothing about distance or order. Four steps: each word writes three cards, a **query** (what I want), a **key** (what I am) and a **value** (what I hand over); **score** every query against every key; **share out** each row of scores so it adds up to 1 (**softmax**); **mix** the values by those shares. Two related words eight apart connect in one step.

1.9 Order, heads, blocks and the bill

Bare attention **cannot see order**: shuffle the words and un-shuffle the outputs, and the difference is **8.88e-16**, rounding error. So every transformer adds **position labels** first. Several **heads** ask several questions at once. A **transformer block** is attention (between words) followed by a small network (within each word), repeated. The bill: the sums grow with the **square** of the length, which is why chatbots had short memories for years.

1.10 The bargain

A true guess built into the wiring substitutes for data. With **little data**, the well-guessed machine wins; with **enormous data**, the machine that guesses less learns the structure itself and then wins. The **Vision Transformer** (a picture cut into squares and treated as words) loses to a convolution model at **1.3 million** pictures and wins at **300 million**: only the data changed. The **bitter lesson**, read properly, is about **where knowledge enters**: as a rule about the answer it tends to lose; as a guess about the shape of the problem it is the bargain. Which side you are on depends on how much data you have.

2 Part 2. The seven questions for anything you design

You may be asked to apply these, so read each one as a test you could run.

#	Question
1	What is connected to what, in one layer?
2	Which dials are shared, and across what: space, time, or nothing?
3	What does that wiring assume, said as a plain sentence about the world?
4	Is that sentence true of my data, and how would I know? (The shuffle test costs one line.)
5	Does my ending keep the guess all the way to the answer?
6	What does it cost, and where is the crossover?
7	How much data do I have, compared with what it would take to learn the structure from scratch?

3 Part 3. Twenty practice questions

Same format as the quiz: one right answer each. Answers on the last page.

1. The sequence 1→2, 2→4, 3→6 fits "double it" but also infinitely many other rules. What does a learner need in order to pick one?

A. More examples, which always settle the question
B. A preference that is not in the data: an inductive bias
C. A larger network
D. A higher learning rate

2. From 128 pictures, a 700-dial model with a built-in guess scores 0.945 on new pictures, and a 16,708-dial model with no guess scores 0.275 (chance is 0.250), despite 1.000 on its training pictures. What is the lesson?

A. A more flexible model is not a better model: with little data, capacity without a guess points nowhere
B. The big model was undertrained
C. Bigger models always need a lower learning rate
D. The test pictures were harder than the training pictures

3. You shuffle the pixels of every picture in the same fixed way. The convolution model falls to chance; the plain model does not move. What does that show?

A. The shuffle damaged the labels
B. The plain model is better in general
C. The convolution model's advantage came entirely from its guess about neighbouring pixels
D. Shuffling always ruins every model

4. What does weight sharing mean in a convolution?

A. Each pixel has its own separate set of weights
B. Weights are copied between different networks
C. The weights are averaged at the end of training
D. The same small kernel is used at every position in the picture

5. An edge kernel's nine numbers add up to zero. What does it output on a perfectly flat grey patch?

A. Zero, because it responds to change, not brightness
B. The brightness of the patch
C. A very large number
D. A negative number, always

6. Which three assumptions does a convolution make about the world?

A. Order, distance and size
B. Locality, stationarity and compositionality
C. Invariance, rotation and colour
D. Sparsity, independence and normality

7. Shift a picture then convolve, or convolve then shift: the difference is 0.00e+00. What property is this, and where does it come from?

A. Invariance, learned during training
B. Overfitting, from too many dials
C. Equivariance, from the arithmetic of sliding one kernel everywhere
D. Regularisation, from the loss function

8. Two models share the same convolution layers. One averages the final map; the other flattens it and wires every position. Trained with the pattern on the left and tested on the right, they score 0.998 and 0.230. What explains the gap?

A. The flattening model needed more epochs
B. The averaging model had more dials
C. Random chance in training
D. The ending decides whether the guess survives: averaging gives invariance; flattening gave every position its own untrained dials

9. A colleague says: "Convolutional networks are simply better for images." On small digit pictures that were already centred and cropped, a plain model scored 0.978 and the convolution model 0.940. What is the right reading?

A. Convolution helps when position still needs solving; here preprocessing had already solved it
B. The convolution model had a bug
C. Digit pictures are not really images
D. The plain model must be overfitting

10. For which data would the convolution guess be the worst fit?

A. Photographs of animals
B. A customer spreadsheet whose column order means nothing
C. Satellite images of fields
D. Handwritten letters that can appear anywhere on the page

11. In a recurrent network, what is the hidden state?

A. A separate network for each word
B. The list of every word seen so far, stored in full
C. A fixed-size notepad the same rulebook updates at every word
D. The final answer

12. A signal is multiplied by the same number at each of 60 steps. With 0.9 it shrinks to about 0.002; with 1.1 it grows to about 304. What is this called?

A. Weight sharing
B. The receptive field
C. Attention
D. The vanishing or exploding gradient

13. What does an LSTM's keep-gate change?

A. The memory is multiplied by a gate chosen at each step and added to, not squashed, so the signal can survive many steps
B. It removes the need for a hidden state
C. It lets the network read all the words at once
D. It makes the notepad unlimited in size

14. At 160 words, a gated reader's signal from the first word still survived, yet it scored at chance. Why?

A. The gates had vanished
B. A surviving signal is necessary but not sufficient: the fixed-size notepad could not hold what the task needed
C. The learning rate was too high
D. Recurrent networks cannot read more than 100 words

15. In attention, what do the query, key and value cards stand for?

A. Position, length and order
B. Input, output and loss
C. What a word wants, what a word is, and what it hands over
D. The three layers of the network

16. After attention scores every query against every key, what does softmax do?

A. Turns each row of scores into shares that add up to 1
B. Removes the smallest scores
C. Sorts the words by importance
D. Adds the position labels

17. Shuffle the words, run bare attention, un-shuffle the outputs: the difference is 8.88e-16. What follows?

A. Attention is broken
B. The model has memorised the sentence
C. Order matters a great deal to attention
D. Bare attention cannot see order, so position labels must be added

18. Why did chatbots have short memories for years?

A. Recurrent networks were still used inside them
B. Attention's cost grows with the square of the length
C. Position labels run out after a few hundred words
D. Softmax cannot handle long rows

19. The Vision Transformer loses to a convolution model at 1.3 million pictures and wins at 300 million. What changed?

A. The design of the transformer
B. The task
C. Only the amount of data
D. The hardware

20. Which reading of the bitter lesson does Week 4 support?

A. Built-in knowledge is useless; only scale matters
B. Scale is useless; only built-in knowledge matters
C. Every problem needs attention
D. It is about where knowledge enters: as a rule about the answer it tends to lose; as a guess about the shape of the problem it is the bargain

4 Answers

#	Answer	Why
1	**B**	Examples alone never choose an answer; every learner has an inductive bias. Point 1.
2	**A**	The big model memorised; it had no reason to prefer the right answer. Point 2.
3	**C**	The shuffle test destroys only what the guess is about, and shows where the advantage came from. Point 2.
4	**D**	The nine numbers never change as the stencil slides. Point 3.
5	**A**	A kernel whose numbers sum to zero gives nothing on a flat region: it detects change. Point 3.
6	**B**	Nearby things belong together; a pattern worth finding here is worth finding everywhere; big things are made of small things. Point 3.
7	**C**	Equivariance is built into the arithmetic, not learned. Point 4.
8	**D**	A guess is worth only what the ending keeps of it. Point 4.
9	**A**	On pre-centred data the guess had nothing left to buy. Point 5.
10	**B**	With no meaningful arrangement, “nearby things belong together” is false. Point 5.
11	**C**	One reader, one fixed-size notepad, the same rulebook every step. Point 6.
12	**D**	Repeated multiplication by one number is arithmetic, not a bug. Point 7.
13	**A**	Gates make forgetting a decision; the notepad is still a fixed size. Point 7.
14	**B**	Gates fix the forgetting, not the notepad’s size. Point 7.
15	**C**	Each word writes three cards; queries are scored against keys, and values are mixed. Point 8.
16	**A**	Share out each row so it sums to 1, then mix the values by those shares. Point 8.
17	**D**	8.88e-16 is rounding error: without position labels, attention sees a bag of words. Point 9.
18	**B**	A path of length 1 between any two words is paid for in sums that grow with the square. Point 9.
19	**C**	Same design, same task: the bargain flips with the amount of data. Point 10.
20	**D**	Read properly, the essay is about where knowledge enters, not whether it matters. Point 10.

Scored under 15? Re-read the points for the questions you missed, then try again tomorrow. The answers say which point each question came from.